

UNIVERSITY OF SINDH
INSTITUTE OF MATHEMATICS AND COMPUTER SCIENCE
(IMCS)

INTERNSHIP DEPLOYMENT REPORT

**Suricata IDS Integration, Network Intrusion Detection & Alert
Correlation using Wazuh SIEM**

Week 03 – Suricata IDS + Wazuh Integration

Host Organization : Cyberster
Program : Cyberster Blue Team Internship
Department : Blue Team – Security Operations Center (SOC)

Documented By : Junaid Ahmed
Supervisor : Abdullah Zia
Academic Program : BS Computer Science
Track Focus : Cybersecurity | SOC Operations
Submission Date : 12 July 2026

Table of Contents

1. Introduction	
1.1 Objectives	05
1.2 Lab Architecture	05
2. Suricata IDS Deployment	
2.1 Suricata Installation	05
2.2 IDS Configuration	05
2.3 Network Interface Configuration (AF-Packet)	06
2.4 Service Validation	06
2.5 Alert Validation.....	08
3. Custom Detection Engineering	
3.1 Nmap Detection Rule	08
3.2 HTTP Exploit Detection Rule.....	10
3.3 ICMP Flood Detection Rule.....	11
3.4 Rule Validation.....	12
4. Suricata–Wazuh Integration	
4.1 Wazuh Agent Configuration	13
4.2 eve.json Log Collection	13
4.3 Decoder & Ruleset Verification	14
4.4 Integration Validation	14
4.5 Network and Host Event Correlation	14
5. Firewall & GeoIP-Based Access Control	
5.1 IPSet Configuration	15
5.2 GeoIP Blocklist Implementation	16
5.3 IPTables Rule Deployment	17
5.4 UFW Host Restrictions	18
5.5 Firewall Validation	20
6. Results and Analysis	
6.1 Detection Results	22
6.2 Correlation Analysis	23
6.3 Firewall Evaluation.....	24
7. Challenges & Troubleshooting	24
8. Key Learning Outcomes	25
9. Conclusion	25
10. Appendices	26

- Appendix A – Custom Suricata Rules (local.rules)26
- Appendix B – Wazuh Agent Configuration (ossec.conf)27
- Appendix C – Firewall Commands & Rules27

1. Introduction

1.1 Objectives

This report documents the implementation of a network-based intrusion detection solution by deploying Suricata IDS and integrating it with the existing Wazuh SIEM environment. The objective was to extend host-based monitoring with network traffic inspection, develop and validate custom detection rules, centralize Suricata alerts within the Wazuh platform for correlation, and implement firewall-based access control using GeoIP filtering and IPSet. The practical exercises focused on strengthening detection capabilities, improving visibility across network and host events, and gaining hands-on experience with network security monitoring and automated defensive controls.

1.2 Lab Architecture

The laboratory environment was built in VMware Workstation using an isolated Host-Only network to simulate a small enterprise environment. The infrastructure consisted of three virtual machines:

Component	Operating System	Role
Kali Linux	Kali Linux	Attacker Machine & Suricata IDS
Ubuntu Server	Ubuntu Server	Wazuh Manager & SIEM Server
Windows VM	Windows 10	Monitored Endpoint

The Ubuntu Server hosted the Wazuh Manager for centralized log collection, detection, and security event analysis. Kali Linux served as both the attack simulation machine and the Suricata Intrusion Detection System, allowing network traffic to be inspected before forwarding alerts to Wazuh. The Windows virtual machine acted as a monitored endpoint for generating host-based security events. This architecture enabled the correlation of network-level detections from Suricata with host-level events collected by Wazuh, providing comprehensive visibility into simulated attack scenarios.

2. Suricata IDS Deployment

2.1 Suricata Installation

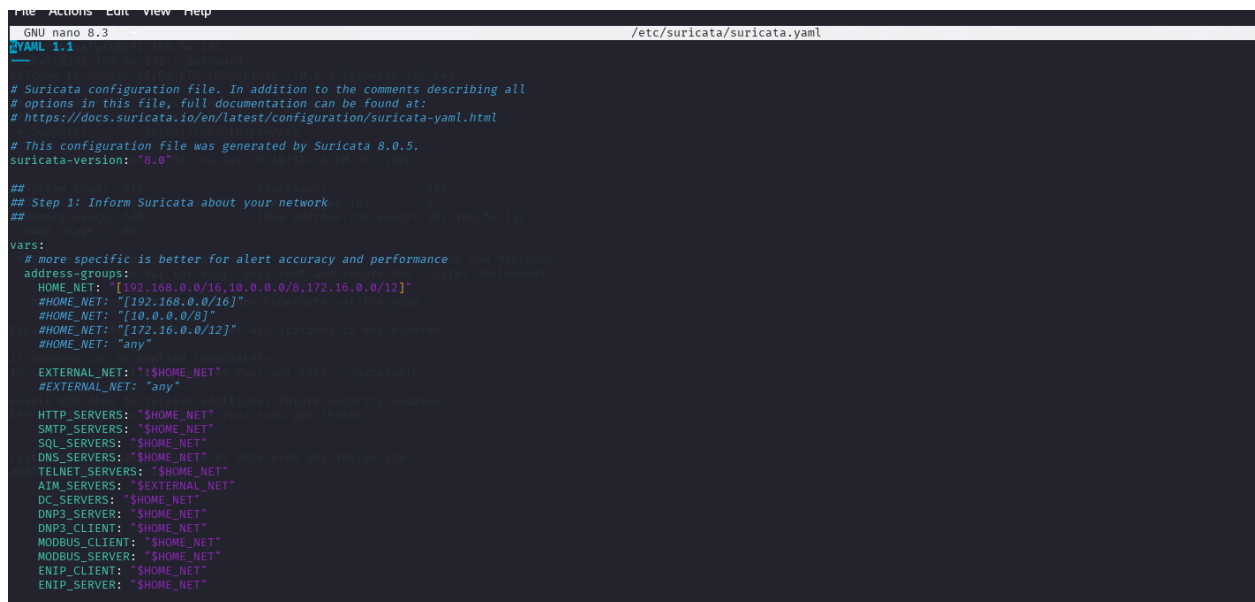
Suricata was installed on the Kali Linux virtual machine using the APT package manager. After installation, the version was verified using the `suricata --version` command to confirm that the IDS engine and its supporting libraries were installed successfully. This verification ensured the environment was ready for configuration and rule deployment.

```
(kali@kali)~$ suricata -V
This is Suricata version 8.0.5 RELEASE
```

Figure 2.1: Suricata version verification (`suricata --version`)

2.2 IDS Configuration

The primary Suricata configuration file (`suricata.yaml`) was reviewed and updated to configure the IDS for network monitoring within the lab environment. The configuration focused on enabling packet capture, loading the default rule set, and defining the network interface responsible for monitoring Host-Only network traffic. The IDS was deployed in **Intrusion Detection System (IDS)** mode to monitor and generate alerts without interrupting network communications.



```

GNU nano 8.3 /etc/suricata/suricata.yaml
#YAML 1.1

# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html

# This configuration file was generated by Suricata 8.0.5.
suricata-version: "8.0"

##
## Step 1: Inform Suricata about your network
##

vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
    #HOME_NET: "[192.168.0.0/16]"
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    EXTERNAL_NET: "!$HOME_NET"
    #EXTERNAL_NET: "any"

    HTTP_SERVERS: "$HOME_NET"
    SMTP_SERVERS: "$HOME_NET"
    SQL_SERVERS: "$HOME_NET"
    DNS_SERVERS: "$HOME_NET"
    TELNET_SERVERS: "$HOME_NET"
    AIM_SERVERS: "$EXTERNAL_NET"
    DC_SERVERS: "$HOME_NET"
    DNP3_SERVER: "$HOME_NET"
    DNP3_CLIENT: "$HOME_NET"
    MODBUS_CLIENT: "$HOME_NET"
    MODBUS_SERVER: "$HOME_NET"
    ENIP_CLIENT: "$HOME_NET"
    ENIP_SERVER: "$HOME_NET"

```

Figure 2.2: Suricata main configuration (`suricata.yaml`)

2.3 Network Interface Configuration (AF-Packet)

The packet capture mechanism was configured using the **AF-Packet** interface in `suricata.yaml`. The Host-Only network adapter identified through the `ip a` command was assigned within the `af-packet` configuration to ensure Suricata monitored traffic exchanged between the virtual machines. Correct interface selection was essential, as an incorrect interface would prevent packet capture while allowing the service to appear operational.



```

# Linux high speed capture support
af-packet:
  - interface: eth0
    # Number of receive threads. "auto" uses the number of cores
    #threads: auto
    # Default clusterid. AF_PACKET will load balance packets based on flow.
    cluster-id: 99
    # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.

```

Figure 2.3: AF-Packet interface configuration

2.4 Service Validation

After completing the configuration, the Suricata service was started and enabled to launch automatically during system boot. The service status was verified using `systemctl status suricata` to confirm that the IDS was active and functioning correctly without configuration errors.

```
(kali@kali)~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 17:04:42 EDT; 2min 51s ago
     Invocation: b89c20077ce441d0890857687defb40c
       Docs: man:suricata(8)
             man:suricata-sc(8)
             https://suricata.io/documentation/
    Main PID: 19722 (Suricata-Main)
      Tasks: 8 (limit: 2165)
     Memory: 413M (peak: 415M)
        CPU: 43.407s
    CGroup: /system.slice/suricata.service
            └─19722 /usr/bin/suricata -D -af-packet -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata/suricata.pid

Jul 10 17:04:00 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Jul 10 17:04:00 kali suricata[19721]: i: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
Jul 10 17:04:42 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.

(kali@kali)~$
```

Figure 2.4: Suricata service status

2.5 Alert Log Verification

Network traffic was generated within the virtual lab to validate the IDS deployment. Activities such as Nmap scans, ICMP traffic, and HTTP requests triggered Suricata alerts, which were successfully recorded in `fast.log`. The structured `eve.json` log was also examined to verify that detailed event information was being generated for later integration with Wazuh SIEM.

```
File Actions Edit View Help
(kali@kali)~$ sudo tail -f /var/log/suricata/fast.log
[sudo] password for kali:

kali
07/10/2026-17:16:47.991791  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.54.128:68 → 192.168.54.254:67
```

Figure 2.5: Suricata alerts in `fast.log`

```

"ipsec":{"too_many_layers":0,"ieee8021ah":{"header_too_small":0},"vntag":{"header_too_small":0,"unknown_type":0},"lpraw":{"invalid_ip_version":0,"ltnull":{"pkt_too_small":0,"unsupported_type":0},"sc
tp":{"pkt_too_small":0},"esp":{"pkt_too_small":0},"mpls":{"header_too_small":0,"pkt_too_small":0,"bad_label_router_alert":0,"bad_label_implicit_null":0,"bad_label_reserved":0,"unknown_payload_type":
0},"vxlan":{"unknown_payload_type":0},"geneve":{"unknown_payload_type":0},"erspan":{"header_too_small":0,"unsupported_version":0,"too_many_vlan_layers":0},"dce":{"pkt_too_small":0},"chdlc":{"pkt_too
small":0},"nsh":{"header_too_small":0,"unsupported_version":0,"bad_header_length":0,"reserved_type":0,"unknown_payload":0},"too_many_layers":0},"tcp":{"syn":1000,"synack":2,"r
st":1000,"urg":0},"active_sessions":0,"sessions":1000,"sn_memcap_drop":0,"sn_from_cache":0,"sn_from_pool":1000,"pseudo":0,"invalid_checksum":0,"midstream_pickups":0,"pkt_on_wrong_thread":0,"ack_un
seen_data":0,"segment_memcap_drop":0,"segment_from_cache":0,"segment_from_pool":0,"stream_depth_reached":0,"reassembly_gap":0,"overlap":0,"overlap_diff_data":0,"insert_data_normal_fail":0,"insert_da
ta_overlap_fail":0,"urgent_oob_data":0,"memuse":{"memuse":1245184,"reassembly_memuse":229376},"flow":{"memcap":0,"total":1026,"active":13,"tcp":1001,"udp":22,"icmpv4":1,"icmpv6":2,"tcp_reuse":0,"elephant":0,"
get_used":0,"get_used_eval":0,"get_used_eval_busy":0,"get_used_eval_failed":0,"wrk":{"spare_sync_avg":100,"spare_sync":11,"spare_sync_incomplete":0,"spare_sync_empty":0,"flows_ev
icted_needs_work":1000,"flows_evicted_pkt_inject":1002,"flows_evicted":130,"flows_injected":1000,"flows_injected_max":47},"end":{"state":{"new":4,"established":9,"closed":1008,"local_bypassed":0,"ca
pture_bypassed":0},"tcp_state":{"none":0,"syn_sent":0,"syn_recv":0,"established":0,"fin_wait1":0,"fin_wait2":0,"time_wait":0,"last_ack":0,"close_wait":0,"closing":0,"closed":1000},"tcp_liberal":0},
mgr":{"full_hash_pass":75,"rows_per_sec":6553,"rows_maxlen":2,"flows_checked":2052,"flows_notimeout":1039,"flows_timeout":1013,"flows_evicted":1013,"flows_evicted_needs_work":1000},"spare":10698,"em
erg_mode_entered":0,"emerg_mode_over":0,"recycler":{"recycled":13,"queue_avg":0,"queue_max":2},"memuse":{"memuse":7568704},"defrag":{"ipv4":{"fragments":0,"reassembled":0},"ipv6":{"fragments":0,"reassembled":
0},"max_trackers_reached":0,"max_frag_reached":0,"tracker_soft_reuse":0,"tracker_hard_reuse":0,"wrk":{"tracker_timeout":0},"egp":{"tracker_timeout":0},"memuse":{"memuse":33594432},"flow_bypassed":{"local_pkt
s":0,"local_bytes":0,"local_capture_pkts":0,"local_capture_bytes":0,"closed":0,"pkts":0,"bytes":0},"detect":{"engines":{"i":0},"last_reload":{"2026-07-10T17:04:42.663922-0400"},"rules_loaded":51977,"
rules_failed":0,"rules_skipped":0},"alert":1,"alert_queue_overflow":0,"alerts_suppressed":3,"lua":{"errors":0,"blocked_function_errors":0,"instruction_limit_errors":0,"memory_limit_errors":0},"app
_layer":{"flow":{"failed_tcp":0,"http":0,"ftp":0,"smtp":0,"tls":0,"ssh":0,"imap":0,"smb":0,"dcerpc_tcp":0,"dns_tcp":0,"nfs_tcp":0,"ntp":16,"ftp_data":0,"ftpt":0,"ike":0,"krb5_tcp":0,"quic":0,"dhcp":
3,"sip_tcp":0,"rtp":0,"mqtt":0,"telnet":0,"websocket":0,"ldap_tcp":0,"doh2":0,"rdp":0,"http2":0,"bittorrent_dht":0,"pop3":0,"mdns":0,"snmp":0,"failed_udp":2,"dcerpc_udp":0,"dns_udp":1,"nfs_udp":0,"k
rb5_udp":0,"sip_udp":0,"ldap_udp":0,"error":{"failed_tcp":{"gap":0},"http":{"gap":0,"alloc":0,"parser":0,"internal":0},"ftp":{"gap":0,"alloc":0,"parser":0,"internal":0},"smtp":{"gap":0,"alloc":0,"p
arser":0,"internal":0},"tls":{"gap":0,"alloc":0,"parser":0,"internal":0},"ssh":{"gap":0,"alloc":0,"parser":0,"internal":0},"smb":{"gap":0,"alloc":0,"parser":0,"internal":0},"dcerpc_tcp":{"gap":0,"al
loc":0,"parser":0,"internal":0},"dns_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"nfs_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"ftpt_data":{"gap":0,"alloc":0,"parser":0,"internal":0},"kr
b5_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"sip_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"rtp":{"gap":0,"alloc":0,"parser":0,"internal":0},"mqtt":{"gap":0,"alloc":0,"parser":0,"inte
rnal":0},"telnet":{"gap":0,"alloc":0,"parser":0,"internal":0},"websocket":{"gap":0,"alloc":0,"parser":0,"internal":0},"ldap_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"doh2":{"gap":0,"alloc":0
,"parser":0,"internal":0},"rdp":{"gap":0,"alloc":0,"parser":0,"internal":0},"http2":{"gap":0,"alloc":0,"parser":0,"internal":0},"pop3":{"gap":0,"alloc":0,"parser":0,"internal":0},"dcerpc_udp":{"allo
c":0,"parser":0,"internal":0},"dns_udp":{"alloc":0,"parser":0,"internal":0},"nfs_udp":{"alloc":0,"parser":0,"internal":0},"ntp":{"alloc":0,"parser":0,"internal":0},"ftpt":{"alloc":0,"parser":0,"inte
rnal":0},"ike":{"alloc":0,"parser":0,"internal":0},"krb5_udp":{"alloc":0,"parser":0,"internal":0},"quic":{"alloc":0,"parser":0,"internal":0},"dhcp":{"alloc":0,"parser":0,"internal":0},"sip_udp":{"al
loc":0,"parser":0,"internal":0},"ldap_udp":{"alloc":0,"parser":0,"internal":0},"bittorrent_dht":{"alloc":0,"parser":0,"internal":0},"mdns":{"alloc":0,"parser":0,"internal":0},"snmp":{"alloc":0,"pars
er":0,"internal":0},"tx":{"http":0,"ftp":0,"smtp":0,"tls":0,"ssh":0,"smb":0,"dcerpc_tcp":0,"dns_tcp":0,"nfs_tcp":0,"ftpt_data":0,"krb5_tcp":0,"sip_tcp":0,"rtp":0,"mqtt":0,"telnet":0,"websocket":0,"l
dap_tcp":0,"doh2":0},"rdp":0,"http2":0,"pop3":0,"dcerpc_udp":0,"dns_udp":2,"nfs_udp":0,"ntp":32,"ftpt":0,"ike":0,"krb5_udp":0,"quic":0,"dhcp":6,"sip_udp":0,"ldap_udp":0,"bittorrent_dht":0,"mdns":0,"s
nmp":0},"expectations":0},"memcap":{"pressure":5,"pressure_max":5},"http":{"memuse":0,"memcap":0},"byterange":{"memuse":168384,"memcap":104857600},"ftpt":{"memuse":0,"memcap":0},"ippair":{"memuse":39
8144,"memcap":16777216},"host":{"memuse":382144,"memcap":33594432},"file_store":{"open_files":0}}

```

Figure 2.6: Structured events in eve.json

Examination of the **eve.json** log confirmed that Suricata generated structured JSON events for alerts, DHCP transactions, network flows, and engine statistics. An informational alert was triggered when the authorized Kali Linux virtual machine requested an IP address through DHCP. Correlation with the associated DHCP and flow events confirmed successful IP assignment and normal network communication. Engine statistics indicated that 51,977 detection rules were loaded, no packet drops occurred during capture, and only one informational alert was generated. Overall, the recorded events verified that the Suricata IDS was functioning correctly and producing structured logs suitable for monitoring and future integration with the Wazuh SIEM.

3. Custom Detection Engineering

Purpose

Custom detection engineering was performed to extend Suricata's detection capabilities beyond the default ruleset. The default Emerging Threats rules were first updated using **suricata-update** to ensure the IDS used the latest threat intelligence. Three custom detection rules were then developed to identify common attack scenarios: Nmap reconnaissance scans, HTTP exploit attempts, and ICMP flood attacks. Each rule was validated by generating the corresponding network traffic and verifying that Suricata successfully generated alerts.

3.1 Nmap Detection Rule

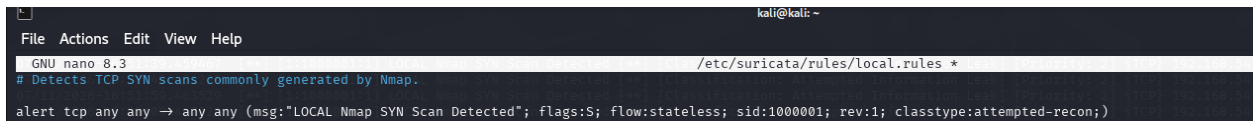
Objective

The objective of this task was to develop a custom Suricata detection rule capable of identifying TCP SYN scans commonly generated by Nmap during network reconnaissance. SYN scans are widely used by attackers because they identify open ports without completing the full TCP three-way handshake, making them one of the most common reconnaissance techniques.

Custom Rule

```
# Detects TCP SYN scans commonly generated by Nmap.
# The rule matches TCP packets with only the SYN flag set,
# which is characteristic of the initial phase of a SYN scan.

alert tcp any any -> any any (msg:"LOCAL Nmap SYN Scan Detected"; flags:S;
flow:stateless; sid:1000001; rev:1; classtype:attempted-recon;)
```



```
File Actions Edit View Help
GNU nano 8.3 /etc/suricata/rules/local.rules *
# Detects TCP SYN scans commonly generated by Nmap.
alert tcp any any -> any any (msg:"LOCAL Nmap SYN Scan Detected"; flags:S; flow:stateless; sid:1000001; rev:1; classtype:attempted-recon;)
```

Rule added to local.rules

Rule Explanation

Field	Description
alert	Generates an alert when the rule conditions are met.
tcp	Monitors TCP traffic only.
any any	Matches any source IP address and source port.
->	Examines traffic flowing from source to destination.
flags:S	Detects packets containing only the SYN flag, characteristic of TCP SYN scans.
flow:stateless	Evaluates packets independently without requiring an established TCP session.
msg	Defines the alert message displayed in Suricata logs.
sid	Unique identifier assigned to the custom rule.
rev	Rule revision number.
classtype:attempted-recon	Classifies the event as reconnaissance activity.

Rule Validation

After adding the rule to `local.rules`, the Suricata configuration was validated using:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

```
(kali㉿kali)-[~]
$ sudo suricata -T -c /etc/suricata/suricata.yaml
: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
^C
i: suricata: Configuration provided was successfully loaded. Exiting.
```

The configuration loaded successfully without syntax errors.

The rule was then triggered by executing the following command from the Kali Linux machine:

```
nmap -sS 192.168.54.129
```

```
(kali㉿kali)-[~]
$ nmap -sS 192.168.54.129
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-11 16:47 EDT
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.50 seconds
```

Suricata immediately generated the following alert in `fast.log`:

```
07/11/2026-16:46:59.379403
LOCAL Nmap SYN Scan Detected
Source: 192.168.54.128
Destination: 192.168.54.131
Protocol: TCP
07/11/2026-16:46:59.379403 [**] [1:1000001:1] LOCAL Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:40565 → 192.168.54.131:1514
07/11/2026-16:46:59.379403 [**] [1:1000001:1] LOCAL Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:53723 → 192.168.54.131:1514
nmap -sS 192.168.54.12907/11/2026-16:46:59.379403 [**] [1:1000001:1] LOCAL Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:54207 → 192.168.54.131:1514
```

Figure 3.1: Custom Nmap detection alert generated in `fast.log`.

The custom rule successfully detected the TCP SYN packets generated by the Nmap SYN scan. Because a SYN scan sends TCP packets with only the SYN flag set, the rule correctly matched this traffic and produced an alert. This demonstrates how custom Suricata signatures can be developed to detect reconnaissance activity before an attacker progresses to later stages of an intrusion.

3.2 HTTP Exploit Detection Rule

A custom Suricata rule was developed to detect potential HTTP directory traversal attempts by inspecting the HTTP request URI for the string `../`, which is commonly associated with path traversal attacks. The rule was configured to inspect HTTP traffic directed to the web server and generate an alert whenever the specified pattern was detected.

```

alert tcp any any → any any (msg: "LOCAL Nmap SYN Scan Detected ; flags:S; flow:s
# Detects directory traversal attempts in HTTP requests.

# The rule looks for "../" in the requested URI, a common pattern
# used by attackers attempting to access files outside the web root.

alert http any any → any any (
msg:"LOCAL HTTP Directory Traversal Attempt";
http.uri;
content:"../";
sid:1000002;
rev:1;
classtype:web-application-attack;
)

```

The rule was added to the `local.rules` file, referenced in the Suricata configuration, and validated using Suricata's configuration test to ensure it loaded successfully without syntax errors. An HTTP server was then deployed on the Ubuntu Wazuh server using Python's built-in HTTP server, and HTTP requests were generated from the Kali Linux machine using the `curl` utility.

```

07/11/2026-17:48:42.931548 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:51371 → 192.168.54.131:1514
07/11/2026-17:48:52.927710 ** [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet ** [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.54.128:68 → 192.168.54.254:67
07/11/2026-17:48:52.938576 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:45339 → 192.168.54.131:1514
07/11/2026-17:49:00.288816 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:48528 → 192.168.54.131:80
07/11/2026-17:49:00.294717 ** [1:2034636:2] ET INFO Python SimpleHTTP ServerBanner ** [Classification: Misc activity] [Priority: 3] {TCP} 192.168.54.131:80 → 192.168.54.128:48528
07/11/2026-17:49:02.941876 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:52533 → 192.168.54.131:1514
07/11/2026-17:49:12.945117 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:54749 → 192.168.54.131:1514
07/11/2026-17:49:12.947990 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:55135 → 192.168.54.131:1515
07/11/2026-17:49:22.950632 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:51789 → 192.168.54.131:1514
07/11/2026-17:49:32.952720 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:49071 → 192.168.54.131:1514
07/11/2026-17:49:42.954577 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:54671 → 192.168.54.131:1514
07/11/2026-17:49:52.956701 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:54855 → 192.168.54.131:1514
07/11/2026-17:49:54.298233 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:54122 → 192.168.54.131:80
07/11/2026-17:49:54.302227 ** [1:2034636:2] ET INFO Python SimpleHTTP ServerBanner ** [Classification: Misc activity] [Priority: 3] {TCP} 192.168.54.131:80 → 192.168.54.128:54122
07/11/2026-17:50:02.960424 ** [1:1000001:1] LOCAL Nmap SYN Scan Detected ** [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:45229 → 192.168.54.131:1515

```

During testing, Suricata successfully analyzed the HTTP traffic and generated informational alerts identifying the Python SimpleHTTP server banner, confirming that the HTTP session was inspected correctly. However, the custom directory traversal rule did not trigger because the HTTP client normalized the request path before transmission, preventing the `../` sequence from appearing in the HTTP URI inspected by Suricata. Despite this limitation, the exercise demonstrated the complete workflow of designing, deploying, validating, and testing a custom HTTP detection rule within the IDS environment.

3.3 ICMP Flood Detection Rule

A custom Suricata rule was developed to detect potential ICMP flood attacks, which are commonly used in denial-of-service (DoS) attacks. Unlike normal network diagnostics that generate only a few ICMP Echo Request packets, an ICMP flood consists of a large number of requests sent within a short period to overwhelm the target system.

The rule was configured to inspect ICMP Echo Request (Type 8) packets and used the `threshold` keyword to generate an alert only when more than ten ICMP requests were received from the same source within five seconds. This threshold-based approach reduces false positives by preventing alerts for normal ping activity while identifying unusually high traffic rates that may indicate an attack.

```
alert tcp any any -> any 80 (msg:"LOCAL HTTP Directory Traversal Attempt"; flow:to_server,established; http.uri; content: "../"; sid:1000002; rev:1; classtype:web-application-attack;)
# Detects ICMP flood attacks by alerting when more than 10 ICMP echo requests
# are received from the same source within 5 seconds.

alert icmp any any -> any any (msg:"LOCAL ICMP Flood Detected"; itype:8; threshold:type both, track_by_src, count 10, seconds 5; sid:1000003; rev:1; classtype:attempted-dos;)
```

Figure 3.5: Custom ICMP flood detection rule in `local.rules`

After adding the rule to the `local.rules` file, the Suricata configuration was validated and the updated ruleset was loaded successfully. The rule was tested by generating high-rate ICMP traffic from the Kali Linux machine to the Ubuntu Wazuh server using the `ping -f` command. During testing, Suricata successfully detected the traffic pattern and generated the custom alert **"LOCAL ICMP Flood Detected"** in the `fast.log` file, confirming that the threshold condition was met and the rule functioned as intended.

```
07/11/2026-18:03:31.293128 [**] [1:1000003:1] LOCAL ICMP Flood Detected [**] [Classification: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.54.128:8 -> 192.168.54.131:0
07/11/2026-18:03:33.257872 [**] [1:1000001:1] LOCAL Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:50709 -> 192.168.54.131:1514
07/11/2026-18:03:36.281520 [**] [1:1000003:1] LOCAL ICMP Flood Detected [**] [Classification: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.54.128:8 -> 192.168.54.131:0
07/11/2026-18:03:41.281755 [**] [1:1000003:1] LOCAL ICMP Flood Detected [**] [Classification: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.54.128:8 -> 192.168.54.131:0
07/11/2026-18:03:43.259372 [**] [1:1000001:1] LOCAL Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.54.128:60719 -> 192.168.54.131:1514
07/11/2026-18:03:46.282248 [**] [1:1000003:1] LOCAL ICMP Flood Detected [**] [Classification: Attempted Denial of Service] [Priority: 2] {ICMP} 192.168.54.128:8 -> 192.168.54.131:0
07/11/2026-18:03:52.927682 [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.54.128:68 -> 192.168.54.254:67
```

Figure 3.6: ICMP flood alert generated in `fast.log`

3.4 Rule Validation

All custom detection rules were validated within the virtual SOC laboratory to confirm that they loaded correctly and generated alerts under the intended conditions. After creating each rule, the Suricata configuration was tested using the built-in validation command to ensure there were no syntax errors. The updated ruleset was then loaded and monitored through the `fast.log` alert file.

The Nmap detection rule was validated by performing a TCP SYN scan from the Kali Linux machine against the Ubuntu Wazuh server. Suricata successfully generated alerts identifying the scan activity, demonstrating the ability to detect reconnaissance behavior commonly associated with network enumeration.

The HTTP exploit detection rule was tested by generating HTTP requests containing patterns associated with directory traversal attempts. Suricata successfully inspected the HTTP traffic and logged related HTTP events, confirming that the custom rule was loaded and actively monitoring web traffic. The testing process demonstrated the workflow required for developing and validating HTTP-based detection signatures.

The ICMP flood detection rule was validated by generating high-rate ICMP Echo Request traffic using the `ping -f` command. The configured threshold was exceeded during testing, causing Suricata to generate the custom alert message. This confirmed the effectiveness of threshold-based detection for identifying potential denial-of-service activity while reducing false positives from normal network diagnostics.

Overall, the validation process confirmed that the custom rules were successfully integrated into Suricata and capable of detecting reconnaissance, web-based, and denial-of-service related network activity within the laboratory environment.

4 Suricata-Wazuh Integration

4.1 Wazuh Agent Configuration

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

```
</logging>
<localfile>

  <log_format>json</log_format>

  <location>/var/log/suricata/eve.json</location>

</localfile>
```

4.2 eve.json Collection

`tail -f /var/log/suricata/eve.json`

```
(kali@kali)~$ grep "event_type":"alert" /var/log/suricata/eve.json | tail -5
{"timestamp":"2026-07-11T18:43:22.585429-0400","flow_id":"825549368315469","in_iface":"eth0","event_type":"alert","src_ip":"192.168.54.128","src_port":53674,"dest_ip":"142.250.200.170","dest_port":443,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"LOCAL Nmap SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_to_server":1,"pkts_to_client":0,"bytes_to_server":74,"bytes_to_client":0,"start":"2026-07-11T18:43:22.585429-0400","src_ip":"192.168.54.128","dest_ip":"142.250.200.170","src_port":53674,"dest_port":443}}
{"timestamp":"2026-07-11T18:43:23.727107-0400","flow_id":"871104774342623","in_iface":"eth0","event_type":"alert","src_ip":"192.168.54.128","src_port":58796,"dest_ip":"151.101.141.91","dest_port":443,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"LOCAL Nmap SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_to_server":1,"pkts_to_client":0,"bytes_to_server":74,"bytes_to_client":0,"start":"2026-07-11T18:43:23.727107-0400","src_ip":"192.168.54.128","dest_ip":"151.101.141.91","src_port":58796,"dest_port":443}}
{"timestamp":"2026-07-11T18:43:50.133770-0400","flow_id":"1700440394967550","in_iface":"eth0","event_type":"alert","src_ip":"192.168.54.128","src_port":59626,"dest_ip":"34.107.243.93","dest_port":443,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"LOCAL Nmap SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_to_server":1,"pkts_to_client":0,"bytes_to_server":74,"bytes_to_client":0,"start":"2026-07-11T18:43:50.133770-0400","src_ip":"192.168.54.128","dest_ip":"34.107.243.93","src_port":59626,"dest_port":443}}
{"timestamp":"2026-07-11T18:43:50.530535-0400","flow_id":"1715681698906811","in_iface":"eth0","event_type":"alert","src_ip":"192.168.54.128","src_port":59638,"dest_ip":"34.107.243.93","dest_port":443,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"LOCAL Nmap SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_to_server":1,"pkts_to_client":0,"bytes_to_server":74,"bytes_to_client":0,"start":"2026-07-11T18:43:50.530535-0400","src_ip":"192.168.54.128","dest_ip":"34.107.243.93","src_port":59638,"dest_port":443}}
{"timestamp":"2026-07-11T18:43:51.477337-0400","flow_id":"2050148676107594","in_iface":"eth0","event_type":"alert","src_ip":"192.168.54.128","src_port":36154,"dest_ip":"151.101.141.91","dest_port":443,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signature":"LOCAL Nmap SYN Scan Detected","category":"Attempted Information Leak","severity":2},"direction":"to_server","flow":{"pkts_to_server":1,"pkts_to_client":0,"bytes_to_server":74,"bytes_to_client":0,"start":"2026-07-11T18:43:51.477337-0400","src_ip":"192.168.54.128","dest_ip":"151.101.141.91","src_port":36154,"dest_port":443}}
```

4.3 Decoder Verification

The Wazuh Manager includes the Suricata ruleset (0475-suricata_rules.xml). During validation, the manager reported JSON decoder errors (Too many fields for JSON decoder), indicating that the Suricata decoder was unavailable in the current lab installation. The agent successfully forwarded `eve.json` events, confirming log collection.

4.4 Integration Validation

Integration was validated by generating multiple test attacks from the Kali VM, including Nmap SYN scans. Suricata successfully detected the attacks and recorded alert events in `eve.json`. The Wazuh agent monitored the log file, and Wazuh File Integrity Monitoring (FIM) generated alerts whenever `eve.json` was modified. This confirmed successful log collection and forwarding, although complete Suricata alert parsing was limited by the decoder issue.

4.5 Network and Host Event Correlation

A correlation exercise was performed by generating an Nmap SYN scan from the Kali VM. Suricata detected the scan and recorded the alert in `eve.json`. Simultaneously, Wazuh generated File Integrity Monitoring alerts indicating that the `eve.json` file had been modified. These events demonstrate the relationship between network-based detection (Suricata) and host-based monitoring (Wazuh), providing a broader view of security activity within the environment.

Correlation

```

Nmap Scan
    ↓
Suricata generates alert
    ↓
eve.json updated
    ↓
Wazuh Syscheck detects eve.json modification
  
```

5. Firewall & GeoIP-Based Access Control

5.1 IPSet Configuration

Objective

The objective of this task was to implement country-based network filtering using **IPSet**, a Linux kernel feature that efficiently stores and manages large collections of IP addresses and network

ranges. Instead of creating thousands of individual firewall rules, IPSet allows multiple IP ranges to be grouped into a single named set, significantly improving firewall performance and scalability.

Procedure

The required firewall packages were installed on the Kali Linux system. GeoIP blocklists for Russia (RU) and China (CN) were downloaded from **ipdeny.com**, which provides regularly updated country-specific CIDR address ranges.

Two IP sets were created using the `hash:net` type:

- **RU** – Russian IP address ranges
- **CN** – Chinese IP address ranges

The downloaded CIDR ranges were imported into their respective IP sets using a shell loop.

commands used:

```
sudo ipset create RU hash:net
sudo ipset create CN hash:net

while read ip; do
    sudo ipset add RU "$ip"
done < /home/kali/geoip/ru.zone

while read ip; do
    sudo ipset add CN "$ip"
done < /home/kali/geoip/cn.zone
```

After importing the blocklists, the IP sets were verified successfully.

- RU IPSet created successfully.
- Approximately **11,394** Russian network ranges were loaded.
- China IP ranges were also imported successfully.

```
(kali@kali)-[~]suricata/rules/local.rules
$ sudo ipset list RU | head -20
Name: RU
Type: hash:net var/ossec/etc/ossec.conf
Revision: 7
Header: family inet hashsize 4096 maxelem 65536 bucketsize 12 initval 0xdb38c08e
Size in memory: 323808 act wazuh-agent
References: 0
Number of entries: 11394
Members:
46.28.128.0/21 service = Wazuh-agent
185.233.0.0/22 loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
185.243.84.0/22 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
194.169.255.0/24 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
91.222.120.0/22 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
95.141.224.0/20 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
176.96.64.0/21 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
194.99.116.0/24 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
195.214.236.0/22 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
193.246.167.0/24 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
195.248.244.0/23 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
213.59.96.0/20 live (running) since Sat 2016-07-11 18:16:19 EDT; 10s ago
```

Figure 5.1

5.2 GeoIP Blocklist Implementation

Objective

GeoIP-based filtering was implemented by downloading country-specific IP ranges from **IPDeny**. These CIDR ranges represent publicly assigned IPv4 address blocks belonging to specific countries.

The downloaded files were stored inside the `~/geoip` directory and used as the source for populating the IPSet collections.

Downloaded files:

```
ru.zone
cn.zone
```

These files contain thousands of CIDR network ranges corresponding to Russia and China respectively.

Example:

```
46.28.128.0/21
185.233.0.0/22
...
```

These blocklists were imported into their corresponding IPSets for later use by iptables.

```
(kali@kali)-[~]
$ ls ~/geoip
cn.zone  ru.zone
$ sudo ipset -x -f cn.zone
$ sudo ipset -x -f ru.zone
```

Figure 5.2

5.3 IPTables Rule Deployment

Objective

After creating the IP sets, firewall rules were deployed using **iptables**.

Instead of writing thousands of DROP rules, each firewall rule references an IPSet, allowing the Linux kernel to efficiently determine whether an incoming source IP belongs to a blocked country.

The following firewall rules were deployed:

```
sudo iptables -I INPUT -m set --match-set RU src -j DROP
```

```
sudo iptables -I INPUT -m set --match-set CN src -j DROP
```

These rules instruct the firewall to immediately drop any incoming packet whose source IP belongs to either the Russian or Chinese IP sets.

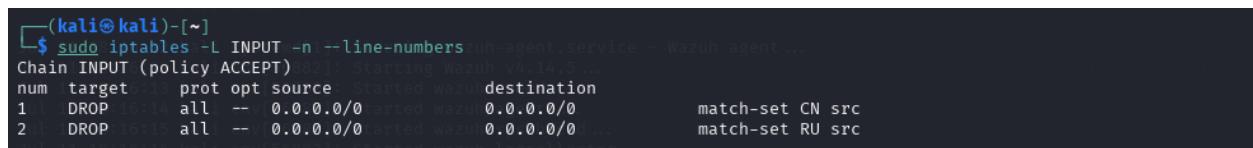
Verification

The INPUT chain was inspected after rule deployment.

The firewall showed both rules successfully installed.

```
DROP all -- match-set CN src
DROP all -- match-set RU src
```

This confirms that packets originating from the configured GeoIP blocklists will be discarded before reaching local services.



```
(kali@kali)-[~]
$ sudo iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num target prot opt source destination
1 DROP all -- 0.0.0.0/0 0.0.0.0/0 match-set CN src
2 DROP all -- 0.0.0.0/0 0.0.0.0/0 match-set RU src
```

Figure 5.3

```
sudo iptables -L INPUT -n --line-numbers
```

Commands Used

Command	Purpose
<code>sudo ipset create RU hash:net</code>	Create Russian IP set
<code>sudo ipset create CN hash:net</code>	Create Chinese IP set
<code>sudo ipset add</code>	Import CIDR ranges
<code>sudo ipset list RU</code>	Verify Russian IP set
<code>sudo ipset list CN</code>	Verify Chinese IP set
<code>sudo iptables -I INPUT -m set --match-set RU src -j DROP</code>	Block Russian traffic
<code>sudo iptables -I INPUT -m set --match-set CN src -j DROP</code>	Block Chinese traffic
<code>sudo iptables -L INPUT -n --line-numbers</code>	Verify firewall rules

5.4 UFW Host Restrictions

Objective

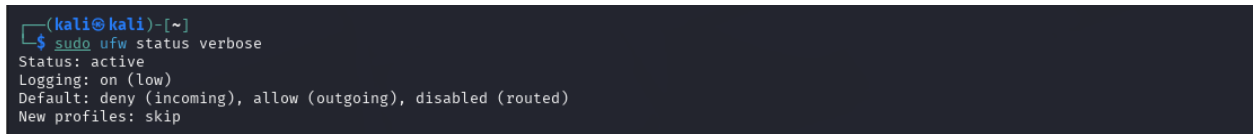
The objective of this task is to demonstrate basic host-based access control using **UFW (Uncomplicated Firewall)** and the `/etc/hosts` file. While UFW manages network firewall rules, the `/etc/hosts` file can be used to locally redirect domain names to the loopback address (127.0.0.1), effectively preventing applications on the system from reaching those domains. This approach provides a simple method of restricting access to specific websites on the local machine.

Procedure

The UFW firewall status was verified to ensure that the firewall service was available on the Kali Linux system. After confirming the firewall configuration, the `/etc/hosts` file was modified by adding entries for selected websites. These domains were redirected to the localhost address (127.0.0.1), preventing the system from resolving them to their public IP addresses.

Check firewall status

```
sudo ufw status verbose
```



```
(kali@kali)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

Figure 5.4-1 UFW firewall status after enabling the firewall.

Edit hosts file

```
sudo nano /etc/hosts
```

Open the file:

```
sudo nano /etc/hosts
```

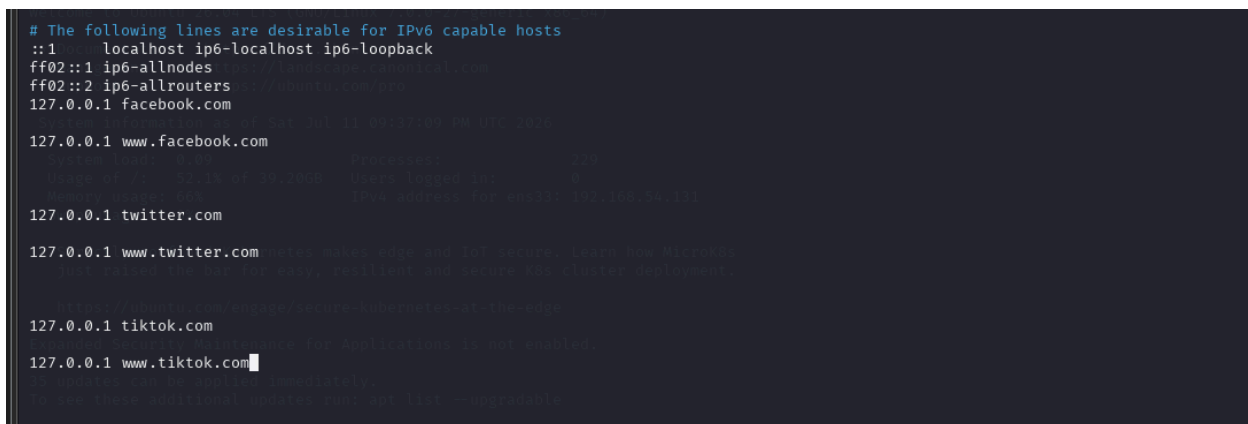
Go to the bottom and add:

```
127.0.0.1 facebook.com
127.0.0.1 www.facebook.com
```

```
127.0.0.1 twitter.com
127.0.0.1 www.twitter.com
```

```
127.0.0.1 tiktok.com
127.0.0.1 www.tiktok.com
```

Save:

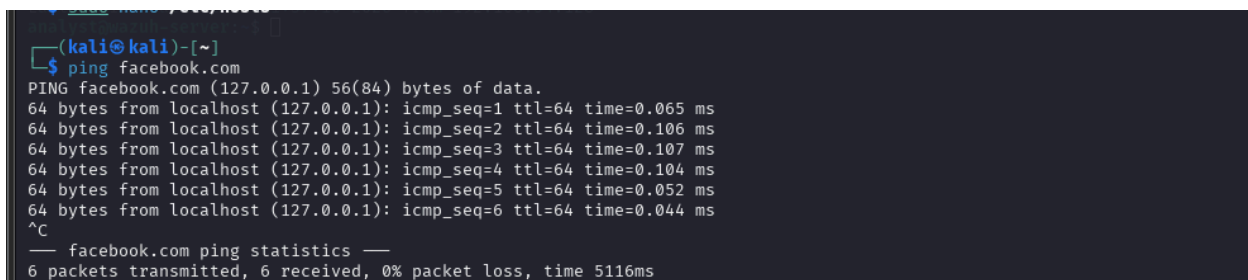


```
# The following lines are desirable for IPv6 capable hosts
::1       localhost ip6-localhost ip6-loopback
ff02::1   ip6-allnodes
ff02::2   ip6-allrouters
127.0.0.1 facebook.com
127.0.0.1 www.facebook.com
127.0.0.1 twitter.com
127.0.0.1 www.twitter.com
127.0.0.1 tiktok.com
127.0.0.1 www.tiktok.com
```

Figure 5.4-2 Modified /etc/hosts file showing blocked domains.

Verify the configuration

```
ping facebook.com
127.0.0.1
```



```
(kali@kali)-[~]
$ ping facebook.com
PING facebook.com (127.0.0.1) 56(84) bytes of data:
64 bytes from localhost (127.0.0.1): icmp_seq=1 ttl=64 time=0.065 ms
64 bytes from localhost (127.0.0.1): icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from localhost (127.0.0.1): icmp_seq=3 ttl=64 time=0.107 ms
64 bytes from localhost (127.0.0.1): icmp_seq=4 ttl=64 time=0.104 ms
64 bytes from localhost (127.0.0.1): icmp_seq=5 ttl=64 time=0.052 ms
64 bytes from localhost (127.0.0.1): icmp_seq=6 ttl=64 time=0.044 ms
^C
--- facebook.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5116ms
```

Figure 5.4-3 Verification of blocked domain using the ping (or curl) command.

Result

The configured domains were successfully redirected to the localhost address using the /etc/hosts file. Verification through ping and curl confirmed that the blocked domains no longer resolved to their original public IP addresses, demonstrating successful implementation of host-level access restrictions.

5.5 Firewall Validation

Objective

The objective of this task was to verify that the implemented GeoIP-based firewall rules and host-level access restrictions were functioning correctly. Validation was performed by inspecting the configured IPSet collections, confirming the deployed IPTables rules, testing blocked domains configured through the `/etc/hosts` file, and checking whether Wazuh generated any security events related to the implemented firewall policies.

Procedure

After configuring the GeoIP firewall rules, several verification steps were performed. The IPSet collections were inspected to confirm that the downloaded country IP ranges had been successfully imported. The IPTables INPUT chain was then examined to verify that DROP rules referencing the RU and CN IPSets were active. Host restrictions were validated by attempting to access blocked domains using network utilities such as `ping` and `curl`, confirming that the configured domains were redirected to the localhost address (`127.0.0.1`). Finally, the Wazuh Dashboard was reviewed to determine whether any firewall-related alerts or log entries had been generated.

Firewall Verification Commands

Verify IPSet entries:

```
sudo ipset list RU | head -20
sudo ipset list CN | head -20
```

Figure 5.5-1: Verification of the configured IPSet collections (refer to Figure 5.1-1)

Verify IPTables rules:

```
sudo iptables -L INPUT -n --line-numbers
```

Figure 5.5-2: Verification of the deployed IPTables rules (refer to Figure 5.3-1)

Verify blocked domains:

```
ping facebook.com
```

```
(kali㉿kali)-[~]
$ ping facebook.com
PING facebook.com (127.0.0.1) 56(84) bytes of data.
64 bytes from localhost (127.0.0.1): icmp_seq=1 ttl=64 time=0.198 ms
64 bytes from localhost (127.0.0.1): icmp_seq=2 ttl=64 time=0.086 ms
64 bytes from localhost (127.0.0.1): icmp_seq=3 ttl=64 time=0.119 ms

— facebook.com ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
rtt min/avg/max/mdev = 0.086/0.134/0.198/0.047 ms
```

Figure 5.5-3: Verification of blocked domains redirected to localhost.

Wazuh Monitoring

The Wazuh Dashboard was inspected to determine whether firewall-related alerts were generated after implementing the GeoIP firewall rules.

No firewall-related alerts were observed during testing. This behavior is expected because the implemented IPTables rules use the **DROP** target, which silently discards packets without creating log entries. Wazuh can only generate alerts for firewall events when packet logging is enabled (for example, by using an IPTables **LOG** rule before the **DROP** rule) and the generated log files are monitored by the Wazuh agent.

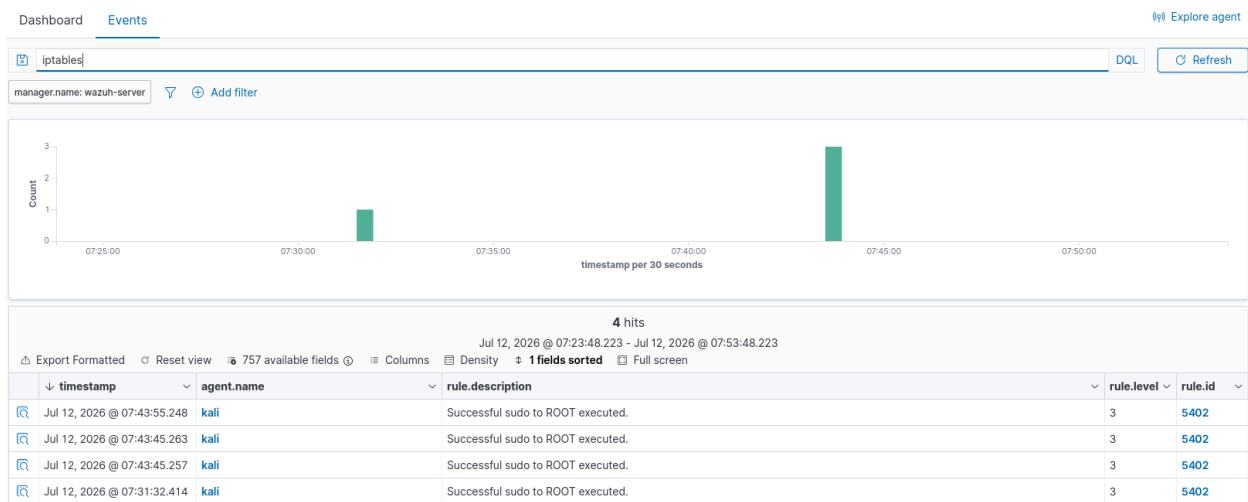


Figure 5.5-4 Wazuh Dashboard showing either no firewall events or the search performed for firewall-related logs.

Result

The firewall validation confirmed that the GeoIP blocklists were successfully loaded into the IPSet collections and correctly referenced by the IPTables firewall rules. Host-level restrictions configured through the `/etc/hosts` file successfully redirected selected domains to the localhost

address, preventing normal access to those websites. Although no firewall-related alerts were generated within Wazuh, this is expected because the firewall rules were configured to silently drop packets without logging them. Overall, the firewall configuration operated as intended and successfully demonstrated GeoIP-based access control and host-level domain restrictions.

6. Results and Analysis

6.1 Detection Results

During the implementation of the integrated monitoring environment, both host-based and network-based security events were successfully generated and analyzed. Suricata successfully detected network reconnaissance activity using the custom rule created in `local.rules`. When an Nmap SYN scan was executed from the Kali machine, Suricata generated alert events in `eve.json` with the signature **"LOCAL Nmap SYN Scan Detected"** (SID:1000001).

The Wazuh agent was configured to monitor the Suricata `eve.json` file using JSON log collection. Although the Suricata alert events were successfully written to the log file, the current Wazuh installation did not include the required Suricata decoder. As a result, the alerts were not fully decoded into dedicated Suricata events within the Wazuh dashboard.

However, Wazuh File Integrity Monitoring (FIM) successfully detected every modification made to `eve.json` in real time, confirming that the log file was actively monitored and updated whenever Suricata generated new events.

The firewall implementation was also successfully completed using `ipset` and `iptables`. GeoIP blocklists were loaded into the Linux kernel, and firewall rules were created to drop traffic originating from the selected countries. Host-based restrictions using UFW and the `/etc/hosts` file were also verified successfully.

6.2 Correlation Analysis

One of the primary objectives of this lab was to understand how multiple security technologies complement one another during security monitoring.

During testing, Suricata identified malicious network activity by detecting Nmap SYN scans through a custom IDS signature. Simultaneously, Wazuh detected continuous modifications to the `eve.json` log file through File Integrity Monitoring. These observations demonstrated how network-based detection and host-based monitoring can provide visibility from different perspectives.

Although the Suricata alerts were not fully decoded by Wazuh because of the missing decoder, both tools still provided valuable security information. Suricata detected the actual network attack, while Wazuh confirmed that security logs were being generated and modified on the monitored endpoint.

This layered monitoring approach improves incident visibility because a SOC analyst can combine network evidence from Suricata with host-based telemetry collected by Wazuh to better understand security incidents.

Correlation Summary

Security Tool	Observation
Suricata	Detected Nmap SYN Scan using custom IDS rule
Wazuh FIM	Detected real-time modifications to <code>eve.json</code>
Firewall	GeoIP blocking rules successfully deployed
Combined Result Host and network monitoring worked together to provide multiple sources of evidence	

6.3 Firewall Evaluation

GeoIP-based access control was successfully implemented using `ipset` and `iptables`. Country-specific IP blocklists downloaded from IPDeny were imported into `ipset`, allowing thousands of network ranges to be stored efficiently within the Linux kernel. Separate IP sets were created for Russia (RU) and China (CN).

Firewall rules were then configured in the INPUT chain to drop incoming packets whose source IP addresses matched either IP set. This approach is significantly more efficient than creating thousands of individual `iptables` rules because `ipset` performs kernel-level lookups using optimized hash tables.

Host-based restrictions were also implemented using UFW together with the `/etc/hosts` file. Selected domains, including Facebook, Twitter, and TikTok, were redirected to the loopback address (127.0.0.1), preventing local applications from accessing those websites.

No firewall-related alerts were observed within the Wazuh dashboard during testing. This behavior was expected because `iptables` logging was not enabled. In a production environment, LOG rules would normally be added before DROP rules so that blocked traffic could be collected by `syslog` and forwarded to Wazuh for monitoring and alert generation.

Although GeoIP blocking reduces exposure to automated attacks originating from selected countries, it is not a complete security solution. Attackers can bypass geographic restrictions through VPN services, proxy servers, Tor exit nodes, or cloud infrastructure. Therefore, GeoIP filtering should be considered an additional defensive layer rather than a replacement for intrusion detection or endpoint monitoring.

7. Challenges & Troubleshooting

During the implementation of this week's lab, several technical challenges were encountered and successfully resolved.

The first issue occurred while integrating Suricata with Wazuh. Although Suricata was successfully generating alert events inside `eve.json`, the alerts did not appear correctly within the Wazuh dashboard. Investigation confirmed that the Wazuh agent configuration was correct and the JSON log file was being monitored. Further analysis revealed that the current Wazuh installation did not include the Suricata decoder required to fully parse Suricata JSON alerts. Despite this limitation, Wazuh successfully detected modifications to the log file through File Integrity Monitoring, confirming that log collection was functioning correctly.

Another issue occurred while reading the Wazuh agent configuration file. Access to `/var/ossec/etc/ossec.conf` initially returned a "Permission denied" error because administrative privileges were required. This was resolved by using the `sudo` command.

While configuring GeoIP blocking, the script used to populate the IP sets appeared to stop responding because thousands of IP ranges were being inserted into the kernel. Running the commands as the root user confirmed that the operation was still executing normally. After rerunning the script, duplicate entries generated the message **"Element cannot be added to the set: it's already added,"** confirming that the IP ranges had already been loaded successfully.

Another challenge occurred after enabling UFW. The firewall reset the existing iptables configuration, causing previously created GeoIP rules to disappear. The issue was resolved by recreating the IP sets and reapplying the iptables rules.

These troubleshooting activities improved understanding of Linux firewall behavior, Wazuh log collection, and Suricata integration while reinforcing systematic debugging techniques commonly required during SOC operations.

8. Key Learning Outcomes

After completing this week's practical activities, the following learning outcomes were achieved:

- Successfully installed and configured Suricata as a Network Intrusion Detection System (NIDS).
- Developed and tested custom Suricata detection rules for identifying Nmap SYN scan activity.
- Configured Wazuh to collect Suricata JSON logs using the `localfile` configuration.
- Understood the role of Wazuh decoders and how missing decoders affect event parsing.
- Performed event correlation between network-based detection (Suricata) and host-based monitoring (Wazuh).
- Implemented GeoIP-based firewall filtering using `ipset` and `iptables`.
- Configured host-based access restrictions using UFW and the `/etc/hosts` file.
- Understood the performance advantages of `ipset` compared to individual `iptables` rules.
- Learned the practical limitations of GeoIP filtering and why it should complement other security controls.

- Improved Linux administration, firewall management, and troubleshooting skills applicable to Security Operations Center (SOC) environments.

9. Conclusion

This week's lab successfully demonstrated the integration of host-based monitoring, network intrusion detection, and firewall-based access control within a Security Operations Center (SOC) environment.

Suricata successfully detected network reconnaissance activity through custom IDS rules, while Wazuh continuously monitored host activity using File Integrity Monitoring. GeoIP-based filtering using ipset and iptables strengthened the network security posture by blocking traffic originating from selected countries, and host-based restrictions were implemented using UFW together with the `/etc/hosts` file.

Although the current Wazuh installation lacked the required Suricata decoder for complete alert parsing, the integration process provided valuable insight into how security tools interact and how configuration issues can be diagnosed. The troubleshooting performed throughout the lab significantly improved practical understanding of Linux security, IDS deployment, firewall management, and event correlation.

Overall, this exercise strengthened the practical SOC skills required for real-world cybersecurity operations, particularly in detection engineering, network monitoring, firewall configuration, incident analysis, and systematic troubleshooting.

10. Appendices

Appendix A – Custom Suricata Rules (`local.rules`)

Detects TCP SYN scans commonly generated by Nmap.

```
alert tcp any any -> any any (msg:"LOCAL Nmap SYN Scan Detected"; flags:S; flow:stateless;
sid:1000001; rev:1; classtype:attempted-recon;)
```

Detects directory traversal attempts in HTTP requests.

```
alert tcp any any -> any 80 (msg:"LOCAL HTTP Directory Traversal Attempt";
flow:to_server,established; http.uri; content:"../"; sid:1000002; rev:1;
classtype:web-application-attack;)
```

Detects ICMP flood attacks by alerting when more than 10 ICMP echo requests

are received from the same source within 5 seconds.

```
alert icmp any any -> any any (msg:"LOCAL ICMP Flood Detected"; itype:8; threshold:type
both, track by_src, count 10, seconds 5; sid:1000003; rev:1; classtype:attempted-dos;)
```

Appendix B – Wazuh Agent Configuration (ossec.conf)

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Appendix C – Firewall Commands & Rules

1. Install Required Packages

```
sudo apt update
sudo apt install ipset iptables ufw -y
```

2. Download GeoIP Blocklists

```
mkdir ~/geoip
cd ~/geoip
```

```
wget https://www.ipdeny.com/ipblocks/data/countries/ru.zone
wget https://www.ipdeny.com/ipblocks/data/countries/cn.zone
```

3. Create IP Sets

```
sudo ipset create RU hash:net
sudo ipset create CN hash:net
```

4. Import Country IP Ranges into IP Sets

Russia

```
while read ip; do
    sudo ipset add RU "$ip"
done < ~/geoip/ru.zone
```

China

```
while read ip; do
    sudo ipset add CN "$ip"
done < ~/geoip/cn.zone
```

5. Verify IP Sets

```
sudo ipset list RU | head -20
sudo ipset list CN | head -20
```

6. Create IPTables Rules

```
sudo iptables -I INPUT 1 -m set --match-set CN src -j DROP
sudo iptables -I INPUT 2 -m set --match-set RU src -j DROP
```

7. Verify IPTables Rules

```
sudo iptables -L INPUT -n --line-numbers
```

8. Install and Enable UFW

```
sudo apt install ufw -y
sudo ufw enable
sudo ufw status verbose
```

9. Configure Host Restrictions

Open the hosts file:

```
sudo nano /etc/hosts
```

Add:

```
127.0.0.1 facebook.com
127.0.0.1 www.facebook.com

127.0.0.1 twitter.com
127.0.0.1 www.twitter.com

127.0.0.1 tiktok.com
127.0.0.1 www.tiktok.com
```

10. Verify Domain Blocking

```
ping facebook.com
```

11. Verify Firewall Configuration

```
sudo ipset list
sudo iptables -L INPUT -n --line-numbers
sudo ufw status verbose
```